Styling elements using the style attribute in JavaScript

Let's now learn how to add CSS styles to elements. This is done by changing the attribute style. For example, to change the color, you need to build the following chain: elem.style.color, and assign the desired color value to it.

Let's look at an example. Let's say we have this element:

```
<p id="elem">text</p>
```

Let's make this element red:

```
let elem = document.querySelector('#elem');
elem.style.color = 'red';
```

⊗jsPmStSA

№ 1

Given a div and a button. On click on the button, set a width, height, and border to the div.

Hyphenated properties in JavaScript

Properties that are hyphenated, such as font-size, are converted to camelCase. That is, font-size will become fontSize:

```
elem.style.fontSize = '20px';
```

⊗jsPmStHP

№ 1

Given a div with text and a button. On click on the button, set the div's font size to 20px, as well as the top border and background.

Styling elements exception in JavaScript

The property float is an exception, as it is special in JavaScript. For it, you should write cssFloat:

```
elem.style.cssFloat = 'right';
```

⊗jsPmStEx

№ 1

Given a list ul and a button. On button click add property float with value *left* to li tags.

Styling with CSS classes in JavaScript

In the previous lesson, we learned how to change the CSS styles of elements by changing the style attribute. More often than not, this is not a good idea. The fact is that if you scatter CSS styles over JavaScript code, it will be problematic to change the design of the site in the future, since you will have to pick the JavaScript code in search of styles hardcoded there.

More convenient for future support would be to add or remove CSS classes to the element, thereby styling them as needed.

Let's look at an example. Let's have a few paragraphs:

```
<p>text1</p>
<p>text2</p>
<p>text3</p>
```

Let's make it so that when you click on any paragraph, this paragraph is painted in some color, for example, in green. To do this, in the CSS file we will make a special class for coloring paragraphs:

```css
.colored {
  color: green;
}
```

The advantage of using a class is that now it will be easy to change the desired color - for this you will just need to make a change to the CSS file, without picking the JavaScript code. This will be especially convenient if you write the JavaScript code, and in the future someone else will style it (perhaps a less qualified person who can only work with CSS).

So, now, after introducing the class, by clicking on any paragraph, you can change its color by simply adding our class to it:

```js
let elems = document.querySelectorAll('p');

for (let elem of elems) {
  elem.addEventListener('click', function() {
    this.classList.add('colored'); // adds the class to the paragraph
  });
}
```

⊗jsPmStCl

№1

Explain why I chose the word colored for the class name, and not the word green, which clearly indicates the color we want.

⊗jsPmStCl

№2

Given a paragraph. Given buttons 'cross out', 'make bold', 'make red'. Let on clicking on each button the given action happens to the paragraph (becomes red, for example).

Advantage of styling with CSS classes in JavaScript

Using classes instead of changing styles directly has another benefit. With a slight movement of the hand, you can make the styles of the elements toggle.

For example, you can make it so that when you first click on a paragraph, it will be painted in a certain color, and when you click again, it will return to its original color. To do this, you just need to change the add method to the toggle method:

```js
let elems = document.querySelectorAll('p');

for (let elem of elems) {
  elem.addEventListener('click', function() {
    this.classList.toggle('colored');
  });
}
```

⊗jsPmStCA

№1

Modify the previous task so that pressing the button again cancels the action of this button.

Styling application in JavaScript

Let's make a button, by clicking on which the element will be either shown or hidden. Let the element be hidden by default (this is implemented using display: none), and it will be shown by adding the class active . This class will be either added or removed using classList.toggle:

```
<button id="button">click me</button>
<div id="elem"></div>
```

```
#elem {
  display: none;
  width: 200px;
  height: 200px;
  border: 1px solid green;
}
#elem.active {
  display: block;
}
```

```
let button = document.querySelector('#button');
let elem = document.querySelector('#elem');

button.addEventListener('click', function() {
  elem.classList.toggle('active');
});
```